

Parameter Estimation from Point-Process Realizations: Feature Computation, Encoding, and Model Architecture

1 Overview

The task (**task: params**) is to regress the generating parameters of a spatial point process directly from a single realization (point cloud). The pipeline generates realizations of a **Thomas cluster process** (a Neyman–Scott process with a Gaussian offspring kernel), computes one of several derived representations of each cloud, encodes that representation into a fixed-size embedding with a modality-specific neural encoder, and regresses the (transformed) process parameters from the embedding with a shared MLP head trained under mean-squared error.

Five representations (“methods”) of the same underlying point cloud are compared, each with its own encoder but the *same* downstream head and loss:

Method key	Feature	File	Encoder
<code>raw_pc</code>	subsampled point coordinates	<code>clouds.pkl</code>	PointNet-style MLP + max-pool
<code>pi_0/pi_1</code>	persistence image (H_0 / H_1)	<code>images.pkl</code>	small CNN
<code>betti_0/betti_1</code>	Betti curve (H_0 / H_1)	<code>betti.pkl</code>	MLP (“StatsEncoder”)
<code>pairwise</code>	pair-distance empirical CDF	<code>features.pkl</code>	MLP (“StatsEncoder”)

The orchestration lives in `scripts/runners/params/run_params.py`, which dispatches to a per-method `Experiment` subclass (`src/cloudforger/nn/experiments/{raw_pc,persistence_image,betti,pairwise}.py`) built on the shared base class in `src/cloudforger/nn/experiments/base.py`. The features themselves are precomputed once by a separate offline pipeline (`scripts/processing/params/pipeline_lib/`) and cached to the pickle files listed above; the NN runner only loads them.

2 The data-generating process

Each cloud is one realization of a Thomas process, a Neyman–Scott cluster process (`src/cloudforger/processes/Thomas.py`) governed by three parameters $\theta = (\lambda_p, \mu, \sigma)$: `parent_intensity` λ_p , `mean_offspring` μ , and `cluster_scale` σ . Sampling proceeds as:

1. Parents $\{c_i\}_{i=1}^{N_p}$ form a homogeneous Poisson process of intensity λ_p on the observation window $W = [0, 1]^d$ expanded by an edge buffer $b = 4\sigma$ (to avoid boundary truncation of clusters), so $N_p \sim \text{Poisson}(\lambda_p \cdot |W_b|)$.
2. Each parent i produces $K_i \sim \text{Poisson}(\mu)$ offspring.
3. Offspring are displaced from their parent by i.i.d. isotropic Gaussian noise, $x_{ij} = c_i + \varepsilon_{ij}$, $\varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2 I_d)$.

4. Offspring falling outside W are discarded; the retained points form the observed cloud $X = \{x_k\}_{k=1}^N \subset [0, 1]^2$.

The parameter grid used to build the dataset (`configs/params/thomas_cloudgen.yaml`) is a 9^3 log-spaced grid over $\lambda_p \in [10, 200]$, $\mu \in [2, 40]$, $\sigma \in [0.005, 0.1]$, repeated 8 times per grid point ($\approx 4,968$ train/test clouds plus a 15% parameter-disjoint adversarial holdout of ≈ 864 clouds, held out by sampling whole parameter vectors rather than individual realizations). This θ is the regression target for every method below.

3 Feature computation

3.1 Raw point cloud (`raw_pc`)

No topological or statistical summary is computed: the network consumes the point set itself. Because clouds have a variable number of points N but the encoder needs a fixed-size batch tensor, each cloud is resampled to a fixed cardinality $n = \text{n_points}$ ($= 512$ by default) at `__getitem__` time (`src/cloudforger/nn/data.py`, `PointCloudDataset`):

$$\tilde{X} = \{x_k : k \sim \text{Unif}\{1, \dots, N\}\}_{k=1}^n, \quad \text{sampled with replacement iff } N < n.$$

This is done fresh on every access (not cached), so with `replace = True` the same cloud yields a different bootstrap resample on each epoch when $N < n$. Empty clouds ($N = 0$) are replaced by an all-zero (n, d) array. The tensor fed to the encoder has shape (n, d) with $d = 2$ for the 2-D experiments.

3.2 Persistence diagrams (`shared substrate for pi and betti`)

Both the persistence-image and Betti-curve features are derived from a persistence diagram computed once per cloud via a **Vietoris–Rips filtration** (`src/cloudforger/tda/filtration/rips.py`, using `ripser`). For a point cloud X with pairwise Euclidean distances $d(x_i, x_j)$, the Rips complex at scale r is

$$R_r(X) = \{\sigma \subseteq X : d(x_i, x_j) \leq r \ \forall x_i, x_j \in \sigma\},$$

and the filtration $\{R_r(X)\}_{r \geq 0}$ induces persistent homology groups $H_k(R_r(X))$ for $k = 0, 1$ (`maxdim=1`). Each connected topological feature is summarized as a birth–death pair $(b, d) \in \mathbb{R}^2$; the collection for dimension k is the diagram $D_k = \{(b_i, d_i)\}$. This is a single call, `ripser(cloud.points, maxdim=1) ["dgms"]`, with no subsampling — the full point set is used for the Rips complex, in contrast to `raw_pc`.

3.3 Persistence images (`pi_0, pi_1`)

Each diagram D_k is rasterized into a fixed 64×64 image (`src/cloudforger/tda/vectorizer/persistence_image.py`). Points are first mapped from birth–death to birth–persistence coordinates, $p_i = d_i - b_i$, then rendered as a weighted sum of isotropic Gaussians on a regular grid $\{(u_a, v_b)\}_{a,b=1}^{64}$ over a fixed `birth_range` $\times$ `pers_range` box:

$$I(u_a, v_b) = \sum_i w(p_i) \exp\left(-\frac{(u_a - b_i)^2 + (v_b - p_i)^2}{2\sigma_{\text{im}}^2}\right), \quad w(p) = p \text{ (linear_weight)},$$

with kernel bandwidth $\sigma_{\text{im}} = 0.05$ (config `vectorization.sigma`), i.e. mild persistence-weighting that zeroes out diagonal (zero-persistence) points and grows linearly with p . Each homology dimension gets its own image and its own calibrated bounds (`MultiChannelImager`): `birth_range` and `pers_range` are *not* inferred per-cloud but fixed once from the 99th percentile of births/persistences pooled over the entire train/test diagram set (`cloudforger.tda.calibration.calibrate`), so a pixel means the same physical scale across every sample. The resulting array is flipped along the persistence axis so persistence increases upward, and stored per-dimension under `image_tensors[dim]` with shape `(n_clouds, 64, 64)`. At load time `PersistenceImageDataset` adds the channel axis, so the tensor entering the encoder is $(1, 64, 64)$.

3.4 Betti curves (`betti_0`, `betti_1`)

The Betti curve counts, at each filtration scale on a fixed grid $r_1 < \dots < r_{128} \in [0, 1]$ (`src/cloudforger/tda/features`), how many k -dimensional features are alive:

$$\beta_k(r_j) = \sum_i \mathbf{1}\{b_i \leq r_j < d_i\}.$$

Pairs with an infinite death are dropped by default (`drop_infinite=True`); `normalize=False` means the curve is left as a raw count rather than divided by the number of intervals. This produces one length-128 vector per cloud per homology dimension, concatenated when more than one dimension is requested (`BettiCurveFeature.vector`); for a single-dimension experiment (`betti_0` or `betti_1`) the encoder input is a length-128 vector.

3.5 Pairwise / correlation features (`pairwise`)

This is a second-order spatial-statistics summary computed directly from the point cloud, independent of persistent homology (`src/cloudforger/stats/pair_dist.py`, `PairDistanceCDF(CloudStatistic)`). Given the cloud's N points, the statistic draws $M = \text{n_samples}$ ($= 5000$) random *ordered pairs* of distinct points (i, j) , $i \neq j$, uniformly with replacement, computes their normalized Euclidean separation

$$r_m = \frac{\|x_i - x_j\|}{\text{diam}(W)}, \quad \text{diam}(W) = \sqrt{\sum_a (\text{high}_a - \text{low}_a)^2},$$

(the region diagonal — a fixed property of the domain, not of the specific cloud, so it does not leak per-cloud information into the normalization), and evaluates the empirical CDF of $\{r_m\}$ on a fixed grid of 64 points in $[0, 1]$:

$$\hat{F}(r_j) = \frac{1}{M} \sum_{m=1}^M \mathbf{1}\{r_m \leq r_j\}.$$

Implementation detail: values are sorted once and the grid is evaluated with `np.searchsorted(values, grid, side="right")`, giving the same $\hat{F}$ as the indicator sum but in $O(M \log M + G)$ instead of $O(MG)$. The per-cloud RNG is seeded as `cloud.seed + 1,000,937`, so the Monte-Carlo pair sample is reproducible per cloud but decorrelated from the cloud-generation RNG stream. The 64-vector $\hat{F}$ is the feature (`CorrelationFeatures.features["pair_distance_cdf"]`); with only one statistic registered, `feature_matrix` has shape `(n_clouds, 64)`.

4 Feature encoding

All encoders (`src/cloudforger/nn/encoders/`) subclass a common `Encoder(nn.Module)` that fixes an output `embedding_dim = 128`; the point is that every method, regardless of input shape, produces a comparable 128-d embedding consumed by the same head (Sec. ??).

4.1 Point cloud $\rightarrow$ embedding: PointNet-style encoder

`PointNetEncoder` (`encoders/point_cloud.py`) applies a *shared, per-point* MLP $\phi_{\text{pt}} : \mathbb{R}^d \rightarrow \mathbb{R}^{256}$ (hidden widths (64, 128, 256), ReLU after each linear layer) to every one of the $n = 512$ points independently — i.e. the same weights are broadcast over the point dimension — and then applies a symmetric (permutation-invariant) pooling operator, coordinate-wise max, to collapse the point axis:

$$h_k = \phi_{\text{pt}}(x_k) \in \mathbb{R}^{256}, \quad z = \max_{k=1, \dots, n} h_k \in \mathbb{R}^{256}, \quad e = W_{\text{proj}} z + b_{\text{proj}} \in \mathbb{R}^{128}.$$

Symmetric pooling (a simplified PointNet, without the input/feature transform networks of the original architecture) guarantees the embedding e is invariant to the arbitrary order in which points are stored/sampled, which matches the fact that a point cloud is a set, not a sequence. Concretely, in code the batch tensor has shape `(B, n, d)`, the MLP is applied via broadcasting over the last axis, and pooling is `features.max(dim=1)`.

4.2 Persistence image $\rightarrow$ embedding: small CNN

`PIEncoder` (`encoders/persistence_image.py`) treats each $1 \times 64 \times 64$ image as a single-channel image and applies three convolutional blocks, each halving spatial resolution:

$$\text{Conv}_{3 \times 3}(1 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_2 \rightarrow \text{Conv}_{3 \times 3}(32 \rightarrow 64) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}_2 \rightarrow \text{Conv}_{3 \times 3}(64 \rightarrow 128) \rightarrow \text{ReLU}$$

followed by global average pooling (`AdaptiveAvgPool2d(1)`) to a 128-vector independent of spatial size, then a linear layer $W_{\text{proj}} \in \mathbb{R}^{128 \times 128}$ to the shared embedding space. Each homology dimension (H_0, H_1) is trained as an entirely separate single-channel model (`in_channels=1`); the two are not fused in this experiment (that fusion is what `models/multi_modal.py` is for, unused here).

4.3 Betti curve / pairwise CDF $\rightarrow$ embedding: MLP (StatsEncoder)

Both fixed-length vector features (length-128 Betti curve, length-64 pair-distance CDF) share the same encoder class (`encoders/stats.py`): a plain MLP with hidden widths (128, 128) and ReLU activations, followed by a linear projection head:

$$h = \text{ReLU}(W_2 \text{ReLU}(W_1 x + b_1) + b_2), \quad e = W_{\text{proj}} h + b_{\text{proj}} \in \mathbb{R}^{128}.$$

There is no architectural inductive bias here beyond a standard feed-forward network — the vector is treated as an unordered/ordered feature bag, not as a sequence or image.

4.4 Optional covariates

If `use_covariates=True`, a per-cloud covariate matrix (shape (N, n_{cov}) , aligned with points) is reduced to its *mean over points* and concatenated *after* encoding, directly onto the embedding before the head: $e' = [e \parallel \bar{c}] \in \mathbb{R}^{128 + n_{\text{cov}}}$ (`CovariateHeadDataset`, `Experiment.run`). Covariates therefore bypass the encoder entirely and are injected as raw statistics, not learned representations.

5 Model head and loss function

5.1 Label preprocessing

Before training, the three raw parameters $\theta = (\lambda_p, \mu, \sigma)$ are transformed per-label (`experiments/base.py`, `_normalize_labels_by_name`):

1. an optional log transform (default: applied to *every* label, since all process parameters here are positive scale-type quantities), $\hat{\theta}_j = \log \theta_j$;
2. per-parameter standardization using the mean/std computed over the *full* train/test dataset (before the train/val/test split), $y_j = (\hat{\theta}_j - m_j)/s_j$, with $s_j \rightarrow 1$ if the empirical std is 0.

The network is trained to predict $y = (y_1, y_2, y_3)$, not θ directly; inverting the transform at evaluation time is $\hat{\theta}_j = \exp(m_j + s_j \hat{y}_j)$ (not shown above, but implied by storing `label_log_mean`/`label_log_std`/`label_t` alongside the checkpoint for later reconstruction).

5.2 Head architecture

Every method funnels its 128-d (or $128+n_{\text{cov}}$ -d) embedding through the *same* regression head class, `ParameterEstimator(nn.Module)` (`nn/heads/paramest.py`):

$$\hat{y} = W_{\text{out}} \text{Dropout}_{0.1}(\text{ReLU}(W_h e' + b_h)) + b_{\text{out}} \in \mathbb{R}^3,$$

i.e. one hidden layer of width 64 (`hidden_dims=(64,)`) with ReLU and dropout $p = 0.1$, then a linear output layer with 3 units (one per Thomas parameter; in general $n_{\text{params}} = \text{labels.shape}[1]$, e.g. 2 for the Matérn process). Encoder and head are composed by `SingleModalModel` (`nn/models/single_modal.py`).
`forward(x, covariates) = head(cat[encoder(x), covariates])`.

5.3 Loss function and optimization

The loss is plain mean squared error over the normalized targets,

$$\mathcal{L}(\hat{y}, y) = \frac{1}{B} \sum_{b=1}^B \|\hat{y}^{(b)} - y^{(b)}\|_2^2 \quad (\text{nn.MSELoss}()),$$

i.e. every one of the (up to 3) regression targets is weighted equally in the loss *after* log/z-score normalization — since normalization equalizes each label’s variance to 1, this keeps parameters on very different natural scales (e.g. $\lambda_p \sim O(10\text{--}200)$ vs. $\sigma \sim O(0.005\text{--}0.1)$) from dominating the gradient. Optimization (`nn/train.py`, `experiments/base.py` `_train_and_eval`):

- optimizer: Adam, learning rate 10^{-3} (`cfg["lr"]`);
- batch size 32, for 500 epochs (`cfg["batch_size"]`, `cfg["n_epochs"]`);
- data split: random 70/15/15 train/val/test (`nn/splits.py`: `train_val_test_split`, seeded by `cfg["seed"]`);
- model selection: after every epoch, validation MSE is checked and the full `state_dict` is checkpointed in memory whenever it improves; the *best-val* weights (not the final-epoch weights) are reloaded before computing the reported test loss;

- an optional adversarial pass evaluates the frozen best model on a held-out set of parameter vectors disjoint from the train/test grid (`adversarial_clouds.pkl` etc.), reusing the same label normalization statistics fit on the train/test set — this measures extrapolation to unseen (λ_p, μ, σ) combinations, not just unseen realizations of seen parameters.

Everything (best `state_dict`, full model, loss curves, label normalization constants, and — if run — adversarial loss) is serialized to `results.pt/model.pt/results.json` under `results/params/{dim}d/{process}/`